C# + .NET + ASP.NET Core Backend Developer Roadmap

Learn what C# actually runs on before learning advanced language features.

Index / Navigation

Phase 1 — C# Foundations
- Topic 1 — .NET and C# Fundamentals
- Topic 2 — Variables, Data Types and Type System
- Topic 3 — Operators and Expressions
- Topic 4 — Conditional Statements and Loops
- Topic 5 — Methods and Parameter Passing
- Topic 6 — Value Types, Reference Types, Boxing and Unboxing
- Topic 7 — Nullability and Null Handling
- Topic 8 — const, readonly, static and Object Lifetime
- Topic 9 — Strings and String Processing
- Topic 10 — Arrays, Tuples and Basic Data Containers
- Topic 11 — Classes, Objects and Encapsulation
- Topic 12 — Constructors and Object Initialization
- Topic 13 — Inheritance
- Topic 14 — Polymorphism
- Topic 15 — Abstraction and Interfaces
- Topic 16 — Composition, Association and Object Relationships
- Topic 17 — Structs, Records and Enums
- Topic 18 — Exception Handling
- Topic 19 — Resource Management and IDisposable
- Topic 20 — Generics
- Topic 21 — Collections
- Topic 22 — Delegates, Action, Func and Predicate
- Topic 23 — Events and Observer-Style Programming
- Topic 24 — LINQ
- Topic 25 — Extension Methods, Partial Types and Anonymous Types
- Topic 26 — Pattern Matching and Modern C\#
- Topic 27 — async, await and Task-Based Programming
- Topic 28 — Threads, Parallelism and Concurrency
- Topic 29 — Concurrent and Immutable Collections
- Topic 30 — CLR, Garbage Collection and Memory Management
- Topic 31 — SOLID and Core Design Principles
- Topic 32 — Dependency Injection and Service Lifetimes
- Topic 33 — Design Patterns
- Topic 34 — .NET Application Architecture
- Topic 35 — Repository, Unit of Work, Service Layer and Mapping Patterns
- Topic 36 — Unit Testing and Testable C\#
- Topic 37 — ASP.NET Core Fundamentals and Application Startup
- Topic 38 — ASP.NET Core Request Pipeline, Middleware, Routing and Error Handling
- Topic 39 — ASP.NET Core Web APIs and REST
- Topic 40 — ASP.NET Core Security, Integration and Production Backend Design

Authentication and Authorization

API Security

HttpClient and External Services

Background and Hosted Services

Backend Performance

Final Backend Design

Phase 1 — C# Foundations

Topic 1 — .NET and C# Fundamentals

Learn what C# actually runs on before learning advanced language features.

SECTION	Subtopics

- What is .NET?
- .NET Framework vs .NET Core vs modern .NET
- C# language
- CLR — Common Language Runtime
- managed code
- unmanaged code
- CIL / IL
- JIT compilation
- compilation process
- assemblies
- DLLs
- executables
- namespaces
- .NET SDK
- .NET Runtime
- `.csproj`
- solution files
- NuGet packages
- `dotnet` CLI
- Debug vs Release
- application execution lifecycle

EXAMPLE / FLOW

```
C# Code
   ->
Compiler
   ->
IL
   ->
CLR
   ->
JIT
   ->
Machine Code
```

Topic 2 — Variables, Data Types and Type System

SECTION Subtopics

- variables
- constants
- identifiers
- built-in types
- `byte`
- `short`
- `int`
- `long`
- `float`
- `double`
- `decimal`
- `bool`
- `char`
- `string`
- `object`
- `DateTime`
- `DateOnly`
- `TimeOnly`
- `Guid`
- `var`
- `dynamic`
- `object`

- type inference
- strong typing
- literals
- default values
- implicit conversion
- explicit conversion
- casting
- `Convert`
- parsing
- `TryParse`

Important comparison to learn:

EXAMPLE / FLOW

```
var
vs
object
vs
dynamic
```

Topic 3 — Operators and Expressions

SECTION Subtopics

- arithmetic operators
- assignment operators
- comparison operators
- logical operators
- bitwise operators
- increment/decrement
- ternary operator
- null-conditional operator `?.`
- null-coalescing operator ??
- null-coalescing assignment ??=
- operator precedence
- expressions
- checked arithmetic
- checked

- `unchecked`

Topic 4 — Conditional Statements and Loops

SECTION Conditions

- `if`
- `else`
- `else if`
- nested conditions
- `switch`
- switch expressions

SECTION Loops

- `for`
- `foreach`
- `while`
- `do while`
- nested loops
- `break`
- `continue`
- `return`

Also learn:

- loop selection
- infinite loops
- loop performance
- modifying collections while iterating

Topic 5 — Methods and Parameter Passing

SECTION Subtopics

- method declaration
- method invocation
- return types
- `void`
- parameters
- arguments

- optional parameters
- named arguments
- default parameters
- method overloading
- local functions
- expression-bodied methods
- `params`
- recursion

Important interview concepts:

- pass by value
- `ref`
- `out`
- `in`

Study carefully:

EXAMPLE / FLOW

```
Normal parameter
ref
out
in
```

and when each should and should not be used.

Topic 6 — Value Types, Reference Types, Boxing and Unboxing

This is one of the most important C# interview topics.

SECTION Subtopics

- value types
- reference types
- copying value types
- copying references
- memory references
- stack concept
- managed heap
- boxing
- unboxing
- performance cost of boxing

- equality
- reference equality
- value equality

Interview questions covered here:

- What is the difference between value types and reference types?
- What is boxing and unboxing?
- What is stack vs heap?

Topic 7 — Nullability and Null Handling

SECTION	Subtopics

- `null`
- nullable value types
- `Nullable<T>`
- `int?`
- nullable reference types
- nullable annotations
- nullable warnings
- `?`
- `!`
- null-forgiving operator
- `??`
- `??=`
- `?.`
- `is null`
- `is not null`
- null-safe code
- nullable DTOs
- nullable domain models

Topic 8 — const, readonly, static and Object Lifetime

This deserves its own topic because these are frequently confused.

SECTION	Subtopics

- `const`

- `readonly`
- `static`
- `static readonly`
- instance fields
- static fields
- instance members
- static members
- static classes
- static methods
- compile-time constants
- runtime initialization

Important comparison:

EXAMPLE / FLOW

```
const
vs
readonly
vs
static readonly
```

Topic 9 — Strings and String Processing

SECTION Subtopics

- string fundamentals
- string immutability
- string equality
- string comparison
- interpolation
- concatenation
- formatting
- `Substring`
- `Split`
- `Replace`
- `Trim`
- searching
- casing

- culture-sensitive comparison
- `StringBuilder`
- string allocations
- `Span<char>` awareness

Interview questions:

- Why are strings immutable?
- When should `StringBuilder` be used?

Topic 10 — Arrays, Tuples and Basic Data Containers

SECTION	Subtopics

- arrays
- declaring arrays
- initializing arrays
- indexing
- one-dimensional arrays
- multidimensional arrays
- jagged arrays
- array iteration
- array copying
- `Array` methods
- ranges
- indexes

Also:

- tuples
- named tuples
- tuple deconstruction
- `KeyValuePair`
- anonymous types

Interview coding begins here:

- find duplicates
- second largest value
- reverse array
- frequency counting

- remove duplicates

Phase 2 — Object-Oriented Programming

Topic 11 — Classes, Objects and Encapsulation

SECTION	Subtopics

- class
- object
- instance
- fields
- properties
- auto-properties
- getters
- setters
- `init`
- required properties
- constructors
- constructor parameters
- object initializers
- access modifiers
- encapsulation
- information hiding

Access modifiers:

- `public`
- `private`
- `protected`
- `internal`
- `protected internal`
- `private protected`

Topic 12 — Constructors and Object Initialization

SECTION	Subtopics

- default constructor
- parameterized constructor

- overloaded constructors
- constructor chaining
- `this`
- `base`
- static constructors
- private constructors
- primary constructors
- initialization order
- object initializers
- required members

Interview questions covered:

- What is constructor chaining?
- What is a static constructor?

Topic 13 — Inheritance

SECTION Subtopics

- base classes
- derived classes
- `base`
- protected members
- inheritance hierarchy
- `virtual`
- `override`
- `new`
- `sealed`
- sealed class
- sealed method

Important comparison:

EXAMPLE / FLOW

```
override
vs
new
```

Also:

EXAMPLE / FLOW

```
inheritance
vs
composition
```

Topic 14 — Polymorphism

SECTION	Subtopics

- polymorphism definition
- compile-time polymorphism
- runtime polymorphism
- method overloading
- method overriding
- virtual dispatch
- base references
- derived objects
- interface polymorphism

Interview questions:

- What is polymorphism?
- Overloading vs overriding
- Compile-time vs runtime polymorphism

Topic 15 — Abstraction and Interfaces

SECTION	Subtopics

- abstraction
- abstract classes
- abstract methods
- interfaces
- interface implementation
- multiple interfaces
- explicit interface implementation
- default interface members
- interface inheritance
- abstract class vs interface
- choosing interfaces correctly

Important questions:

- Can a class implement multiple interfaces?
- Can interfaces contain implementations?
- Abstract class vs interface?

Topic 16 — Composition, Association and Object Relationships

SECTION	Subtopics

- association
- aggregation
- composition
- inheritance vs composition
- "has-a" relationship
- "is-a" relationship
- dependency relationships
- object collaboration
- composition over inheritance

Important interview topic:

REMEMBER	Why is composition often preferred over inheritance?

Topic 17 — Structs, Records and Enums

SECTION	Subtopics

- `struct`
- class vs struct
- readonly structs
- record classes
- record structs
- positional records
- value equality
- reference equality
- `with`
- immutable models
- enums
- flags enums

Interview question:

REMEMBER	How are records different from classes?

Phase 3 — Core C# Features

Topic 18 — Exception Handling

SECTION	Subtopics

- exceptions
- exception hierarchy
- `try`
- `catch`
- multiple catch blocks
- `finally`
- `throw`
- `throw;`
- custom exceptions
- inner exceptions
- exception filters
- stack trace
- when to throw
- when not to catch
- domain exceptions
- validation vs exceptions
- logging exceptions
- exception best practices

Later this connects directly to ASP.NET global exception handling.

Topic 19 — Resource Management and IDisposable

SECTION	Subtopics

- unmanaged resources
- managed resources
- `IDisposable`
- `Dispose`

- `using`
- using declarations
- `IAsyncDisposable`
- `await using`
- finalizers
- GC finalization
- Dispose pattern

Important comparison:

EXAMPLE / FLOW

```
Dispose
vs
Finalizer
```

Topic 20 — Generics

Generics should be learned before collections and advanced repositories.

SECTION Subtopics

- generic classes
- generic methods
- generic interfaces
- generic delegates
- generic type parameters
- multiple type parameters
- type safety
- generic constraints

Constraints:

- `where T : class`
- `where T : struct`
- `where T : notnull`
- `where T : new()`
- interface constraints
- base-class constraints

Advanced:

- covariance

- contravariance
- `in`
- `out`
- open generics
- generic DI registration

Your project patterns:

EXAMPLE / FLOW

```
IRepository<T>
GenericRepository<T>
```

belong here.

Topic 21 — Collections

Study this deeply.

SECTION Core collections

- `Array`
- `List<T>`
- `Dictionary<TKey,TValue>`
- `HashSet<T>`
- `Queue<T>`
- `Stack<T>`
- `LinkedList<T>`
- `SortedSet<T>`
- `SortedDictionary<TKey,TValue>`
- `PriorityQueue<TElement,TPriority>`

SECTION Interfaces

- `IEnumerable<T>`
- `ICollection<T>`
- `IList<T>`
- `IReadOnlyCollection<T>`
- `IReadOnlyList<T>`
- `IDictionary<TKey,TValue>`

Important comparisons:

EXAMPLE / FLOW

```
Array vs List<T>

IEnumerable
vs
ICollection
vs
IList
```

Also study:

- hashing
- equality comparers
- collection complexity
- lookup performance
- mutable vs immutable collections

Topic 22 — Delegates, Action, Func and Predicate

SECTION Subtopics

- delegates
- delegate instances
- method references
- multicast delegates
- anonymous methods
- lambda expressions

Built-in delegates:

- `Action`
- `Action<T>`
- `Func<T>`
- `Func<T,TResult>`
- `Predicate<T>`

Important interview comparison:

EXAMPLE / FLOW

```
Action
vs
Func
vs
Predicate
```

Topic 23 — Events and Observer-Style Programming

SECTION	Subtopics

- events
- publishers
- subscribers
- `event`
- event handlers
- `EventArgs`
- custom event arguments
- event lifecycle
- memory leaks from subscriptions
- events vs delegates

This connects to the Observer Pattern you identified in event-driven email auditing.

Topic 24 — LINQ

This is one of the most important professional C# topics.

SECTION	Subtopics

- LINQ fundamentals
- method syntax
- query syntax
- deferred execution
- immediate execution

Operators:

- `Where`
- `Select`
- `SelectMany`
- `OrderBy`
- `OrderByDescending`
- `ThenBy`
- `GroupBy`
- `Join`
- `GroupJoin`
- `Any`

- `All`
- `Contains`
- `First`
- `FirstOrDefault`
- `Single`
- `SingleOrDefault`
- `Count`
- `Sum`
- `Average`
- `Min`
- `Max`
- `Distinct`
- `Union`
- `Intersect`
- `Except`
- `Skip`
- `Take`
- `ToList`
- `ToDictionary`

Important:

- `IEnumerable<T>`
- `IQueryable<T>`
- deferred execution
- multiple enumeration
- LINQ performance

Practical interview exercises:

- sort objects
- group records
- remove duplicates
- frequency counts
- find duplicate records
- project DTOs

Topic 25 — Extension Methods, Partial Types and Anonymous Types

SECTION	Subtopics

- extension methods
- extension member concepts
- creating extension methods
- extension method resolution
- good/bad extension methods
- partial classes
- partial methods
- partial members
- anonymous types
- anonymous objects
- compiler-generated types

C# 14 also expands extension-member capabilities. Microsoft Learn

Topic 26 — Pattern Matching and Modern C\#

SECTION	Subtopics

- `is`
- type patterns
- declaration patterns
- constant patterns
- relational patterns
- logical patterns
- property patterns
- positional patterns
- list patterns
- switch expressions
- recursive patterns
- pattern guards

Also include modern language features:

- collection expressions
- raw strings
- file-scoped namespaces

- global usings
- primary constructors
- required members
- field-backed properties
- null-conditional assignment
- modern extension members

Phase 4 — Async, Threading and Runtime

Topic 27 — async, await and Task-Based Programming

SECTION Subtopics

- synchronous vs asynchronous
- CPU-bound vs I/O-bound
- `Task`
- `Task<T>`
- `async`
- `await`
- async method lifecycle
- async all the way
- `Task.Delay`
- cancellation
- `CancellationToken`
- exceptions in async code
- `Task.WhenAll`
- `Task.WhenAny`
- async streams
- `IAsyncEnumerable<T>`
- `ValueTask`
- common async mistakes
- `.Result`
- `.Wait()`
- deadlock concepts

This is mandatory before ASP.NET Core database/API programming.

Topic 28 — Threads, Parallelism and Concurrency

Async and parallel programming are not the same thing.

SECTION	Subtopics

- process
- thread
- thread pool
- concurrency
- parallelism
- CPU-bound work
- `Thread`
- `Task.Run`
- `Parallel.For`
- `Parallel.ForEach`
- PLINQ
- race conditions
- shared state
- synchronization
- `lock`
- `Monitor`
- `SemaphoreSlim`
- atomic operations
- `Interlocked`
- deadlocks
- thread safety

Topic 29 — Concurrent and Immutable Collections

SECTION	Subtopics

- `ConcurrentDictionary`
- `ConcurrentQueue`
- `ConcurrentStack`
- `ConcurrentBag`
- blocking concepts
- immutable collections

- thread-safe collection design
- collection synchronization
- producer/consumer patterns

Also:

- Channels
- `Channel<T>`
- background queues

Topic 30 — CLR, Garbage Collection and Memory Management

Now study the runtime in greater depth.

SECTION	Subtopics

- CLR architecture
- managed heap
- stack frames
- GC roots
- garbage collection
- GC generations

Generations:

- Generation 0
- Generation 1
- Generation 2

Also:

- Large Object Heap
- object allocation
- object lifetime
- compacting GC
- workstation GC
- server GC
- finalization queue
- weak references
- memory pressure
- allocations
- boxing overhead

- string allocations
- Span<T>
- Memory<T>
- pooling
- ArrayPool<T>

Phase 5 — Software Design

Topic 31 — SOLID and Core Design Principles

Study every principle with bad and corrected examples.

SECTION	SOLID

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Also:

- DRY
- KISS
- YAGNI
- separation of concerns
- loose coupling
- high cohesion
- composition over inheritance
- program to abstractions
- explicit dependencies

Your interview questions about large service classes, loose coupling, high cohesion, SRP violations, and dependency inversion all belong here.

Topic 32 — Dependency Injection and Service Lifetimes

First learn DI as a design principle, then ASP.NET Core DI.

SECTION	Concepts

- Dependency Inversion
- Inversion of Control

- Dependency Injection
- constructor injection
- method injection
- property injection
- service locator anti-pattern
- DI containers

ASP.NET Core lifetimes:

- `AddTransient`
- `AddScoped`
- `AddSingleton`

Study these very carefully.

EXAMPLE / FLOW

```
Transient
-> new instance when requested

Scoped
-> one instance per request/scope

Singleton
-> one instance for application lifetime
```

Also:

- captive dependency
- singleton consuming scoped service
- service scopes
- `IServiceScope`
- disposal
- thread safety
- open generic registrations

ASP.NET Core includes a built-in DI system. Microsoft Learn

Topic 33 — Design Patterns

Instead of memorizing them randomly, study them grouped by problem.

Creational

- Factory Method
- Simple Factory
- Abstract Factory

- Builder
- Singleton
- Prototype

Your projects:

- `ExceptionResultFactory`
- `DbContextFactory`
- `WebApplicationBuilder`
- singleton registrations

Structural

- Adapter
- Decorator
- Facade
- Proxy
- Composite
- Bridge

Your projects:

- `HttpClientAbstract` → Adapter-style role
- `EmailServiceDecorator`
- service interfaces → facade-style boundaries
- lazy repository initialization → proxy-like behavior

Behavioral

- Strategy
- Template Method
- Chain of Responsibility
- Observer
- Command
- Mediator
- State

Your projects:

- `ICustomExceptionProvider` → Strategy
- `BaseFunction` → Template Method
- middleware → Chain of Responsibility
- email auditing → Observer

Phase 6 — Architecture and Testing

Topic 34 — .NET Application Architecture

SECTION Subtopics

- separation into layers
- presentation layer
- application layer
- domain layer
- infrastructure layer
- layered architecture
- N-tier
- Clean Architecture
- Onion Architecture
- Hexagonal Architecture
- Ports and Adapters
- vertical slices
- modular monolith
- dependency rule
- domain services
- application services
- repositories
- Unit of Work
- transaction boundaries
- service layer
- DTO pattern
- mapper pattern

Also study architecture trade-offs:

EXAMPLE / FLOW

```
Simple application
-> simple architecture

Complex domain
-> stronger boundaries
```

Do not apply Clean Architecture mechanically to every small API.

Topic 35 — Repository, Unit of Work, Service Layer and Mapping Patterns

These deserve special treatment because they appear heavily in your projects.

SECTION Repository

- repository responsibility
- generic repository
- specific repository
- repositories with EF Core
- repository benefits
- repository over-abstraction risks

SECTION Unit of Work

- transaction coordination
- repositories inside Unit of Work
- EF Core `DbContext` as Unit-of-Work-like abstraction
- when custom UoW is useful
- when it becomes unnecessary

SECTION Service Layer

- business logic boundaries
- orchestration
- avoiding god services
- domain logic vs service logic

SECTION DTO

- request DTO
- response DTO
- entity vs DTO

SECTION Mapper

- manual mapping
- AutoMapper
- profiles
- projection

Your examples:

EXAMPLE / FLOW

```
GenericRepository<T>
IRepository<T>

UnitOfWork
IUnitOfWork

AdminServiceProfile
ConsentServiceProfile
```

Topic 36 — Unit Testing and Testable C\#

SECTION Subtopics

- unit testing
- test isolation
- Arrange / Act / Assert
- xUnit
- test fixtures
- theories
- test data
- mocking
- stubbing
- fakes
- Moq / NSubstitute concepts
- dependency injection for testability
- testing services
- testing async code
- testing exceptions
- testing collections
- testing LINQ logic

Also:

- integration testing concepts
- `WebApplicationFactory`
- API tests
- contract tests

Phase 7 — ASP.NET Core

Topic 37 — ASP.NET Core Fundamentals and Application Startup

SECTION Subtopics

- ASP.NET Core architecture
- `Program.cs`
- `WebApplicationBuilder`
- Host
- Kestrel
- dependency injection
- configuration
- `appsettings.json`
- environment-specific settings
- environment variables
- secrets
- configuration providers
- logging
- environments
- development vs production

Options:

- `IOptions<T>`
- `IOptionsSnapshot<T>`
- `IOptionsMonitor<T>`
- Options pattern
- configuration binding
- validation

ASP.NET Core's current fundamentals explicitly center on `Program.cs`, DI, configuration, logging, routing, middleware, and error handling. Microsoft Learn

Topic 38 — ASP.NET Core Request Pipeline, Middleware, Routing and Error Handling

This should be one of your strongest ASP.NET Core topics.

SECTION Middleware

- request pipeline
- middleware execution

- custom middleware
- `RequestDelegate`
- `HttpContext`
- `Use`
- `Run`
- `Map`
- middleware ordering
- short-circuiting
- conventional middleware
- `IMiddleware`

Flow:

EXAMPLE / FLOW

```
Request
   ->
Exception Handling
   ->
Authentication
   ->
Authorization
   ->
Routing
   ->
Endpoint
   ->
Response
```

SECTION Routing

- endpoint routing
- route templates
- route parameters
- constraints
- attribute routing

SECTION Exception handling

- global exception handling
- exception middleware
- `IExceptionHandler`
- Problem Details
- custom exception mapping

- logging errors
- validation errors

Important DI rule: conventional middleware is created once for the app lifetime, so scoped services normally belong in `InvokeAsync` rather than its constructor unless using factory-based middleware. Microsoft Learn

Topic 39 — ASP.NET Core Web APIs and REST

This combines the interview questions you listed into the correct learning topic.

Web API fundamentals

- controllers
- `ControllerBase`
- `[ApiController]`
- actions
- Minimal APIs
- controller APIs vs Minimal APIs
- routing
- model binding
- model validation

HTTP

- request
- response
- headers
- body
- HTTP methods

Learn:

- GET
- POST
- PUT
- PATCH
- DELETE

REST

- resources
- URIs
- statelessness
- representations

- uniform interface
- REST vs SOAP
- idempotency

Status codes

- `200 OK`
- `201 Created`
- `202 Accepted`
- `204 No Content`
- `400 Bad Request`
- `401 Unauthorized`
- `403 Forbidden`
- `404 Not Found`
- `409 Conflict`
- `422`
- `429 Too Many Requests`
- `500 Internal Server Error`

API contracts

- DTOs
- request models
- response models
- entities vs DTOs
- API contract stability
- serialization
- content negotiation

Production API features

- pagination
- filtering
- sorting
- searching
- rate limiting
- OpenAPI
- Swagger
- API versioning

- backward compatibility
- deprecation

ASP.NET Core 10 continues to provide first-party OpenAPI support. Microsoft Learn

Topic 40 — ASP.NET Core Security, Integration and Production Backend Design

This is the final topic because it combines all earlier concepts.

Authentication and Authorization

- authentication vs authorization
- claims
- roles
- policies
- policy-based authorization
- JWT
- bearer authentication
- token anatomy
- issuer
- audience
- expiration
- signing
- refresh-token concepts
- authentication schemes
- policy schemes

Your Auth Broker pattern:

EXAMPLE / FLOW

```
Policy Scheme Pattern
```

belongs here.

API Security

- HTTPS
- CORS
- CSRF
- XSS awareness

- SQL injection awareness
- validation
- authentication
- authorization
- secrets
- sensitive information
- secure logging
- rate limiting
- least privilege

HttpClient and External Services

- `HttpClient`
- `IHttpClientFactory`
- named clients
- typed clients
- `DelegatingHandler`
- timeout
- retries
- cancellation
- resilience
- circuit breaker concepts

Your project:

EXAMPLE / FLOW

```
DelegatingHandler
-> Decorator-style HTTP pipeline
```

Background and Hosted Services

- `IHostedService`
- `BackgroundService`
- scoped dependencies in background services
- cancellation
- graceful shutdown
- queues

- Channels

Backend Performance

- async I/O
- caching
- memory caching
- distributed caching awareness
- response/output caching
- allocations
- logging performance
- profiling
- thread pool starvation
- database-call efficiency

Final Backend Design

Learn how everything fits:

EXAMPLE / FLOW

```
HTTP Request
   ->
Middleware
   ->
Authentication
   ->
Authorization
   ->
Controller / Endpoint
   ->
Validation
   ->
Service Layer
   ->
Domain Logic
   ->
Repository / Data Access
   ->
Result
   ->
DTO Mapping
   ->
HTTP Response
```

Architecture around it:

EXAMPLE / FLOW

```
API
   ->
Application
   ->
Domain

Infrastructure
   ->
implements external dependencies
```

Where Your EPP-API Patterns Fit

Your pattern list is useful. I would not study those patterns before C#, OOP, SOLID, generics, and DI. Learn them after those foundations.

Pattern from your projects	Roadmap topic
Factory	Topic 33
Singleton	Topics 32–33
Builder	Topic 33 / ASP.NET startup
Repository	Topic 35
Unit of Work	Topic 35
Decorator	Topic 33
Adapter	Topic 33
Facade	Topic 33
Proxy	Topic 33
Strategy	Topic 33
Template Method	Topic 33
Chain of Responsibility	Topics 33 & 38
Observer	Topics 23 & 33
Dependency Injection	Topic 32
Service Layer	Topics 34–35
DTO	Topics 35 & 39
Mapper	Topic 35
Options Pattern	Topic 37
MVC	Topic 39

Pattern from your projects	Roadmap topic
Policy Scheme	Topic 40
Exception Handler	Topics 18 & 38
Middleware Pattern	Topic 38

Interview Coding Practice Should Run in Parallel

Do not wait until Topic 40 to start coding problems.

After Topics 1–10, practice:

- reverse string without built-in reverse;
- palindrome;
- duplicate array values;
- second largest number;
- first non-repeating character;
- frequency counting;
- remove duplicates.

After Collections + LINQ, practice:

- group records;
- sort objects;
- dictionary frequency problems;
- collection transformations;
- `GroupBy`;
- joins;
- projections.

After ASP.NET Core, practice:

- CRUD API;
- custom middleware;
- exception middleware;
- repository/service implementation;
- scoped/transient/singleton examples;
- JWT authentication;
- pagination;
- filtering/sorting;

- API versioning;
- unit tests with mocks.

Correct Learning Order at a Glance

EXAMPLE / FLOW

```
C# Fundamentals
   ->
Type System
   ->
Methods
   ->
Value / Reference Types
   ->
Strings / Arrays
   ->
OOP
   ->
Exceptions
   ->
Generics
   ->
Collections
   ->
Delegates / Events
   ->
LINQ
   ->
Async / Await
   ->
Concurrency / Parallelism
   ->
CLR / Memory
   ->
SOLID
   ->
Dependency Injection
   ->
Design Patterns
   ->
Architecture
   ->
Testing
   ->
ASP.NET Core Fundamentals
   ->
DI Lifetimes
   ->
Middleware Pipeline
   ->
REST APIs
   ->
Validation / DTOs
   ->
Error Handling
   ->
Security / JWT
   ->
API Versioning
   ->
Production Backend Design
```

That is the 40-topic roadmap I recommend for you. It contains the C# interview topics you pasted, the ASP.NET Core topics, DI lifetimes (`Transient`, `Scoped`, `Singleton`), async/parallel programming, runtime/GC, generics, collections, LINQ, OOP, SOLID, architecture, and the actual

design patterns found in your EPP-API and Auth Broker projects—without mixing in Angular, Azure, or SQL Server as separate learning tracks.